Министерство науки и высшего образования РФ
Факультет вычислительной техники
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №2
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Оценка времени выполнения задач»

Выполнили: ст. гр. 25BBB2
Денисов Р.С.
Никоноров С.Д.
Проверили: к.т.н., доцент
Деев М.В.
к.т.н., доцент
Юрова О.В.

Пенза 2026

Название: Оценка времени выполнения программ

Цель работы: научиться оценивать время выполнения программ и сопоставлять результаты измерений с теоретической оценкой сложности алгоритма.

Лабораторные задания: 1) вычислить порядок сложности программы, вычисляющей произведение двух матриц; 2) оценить время выполнения этой программы для матриц размерами 100, 200, 400, 1000, 2000, 4000, 10000 и сравнить результат с теоретической оценкой; 3) оценить время работы сортировки Шелла, реализованной быстрой сортировки и стандартной функции qsort на случайном наборе значений, на возрастающей последовательности, на убывающей последовательности и на последовательности, одна половина которой возрастает, а другая убывает.

Средства измерения: программы реализованы на языке Си. Время выполнения измерялось функцией clock() из библиотеки time.h, которая возвращает процессорное время в тактах с начала работы программы; перевод в секунды выполняется делением на константу CLOCKS_PER_SEC.

Задание 1. Оценка времени выполнения программы умножения матриц

Теоретическая оценка порядка сложности

Программа состоит из двух этапов: заполнения двух исходных матриц случайными числами и собственно перемножения. Заполнение выполняется двумя вложенными циклами по N, то есть требует порядка N^2 операций для каждой матрицы.

Перемножение реализовано тремя вложенными циклами: внешний перебирает строки результирующей матрицы, средний — её столбцы, внутренний накапливает скалярное произведение строки первой матрицы на столбец второй. Каждый из трёх циклов имеет сложность O(N), поэтому сложность всей программы равна произведению $O(N) \cdot O(N) \cdot O(N) = O(N^3)$. Этап заполнения матриц, имеющий порядок $O(N^2)$, при больших N даёт пренебрежимо малый вклад по сравнению с перемножением.

На рисунке 1 показан ожидаемый вид зависимости времени выполнения от размера матрицы N при кубической сложности — до проведения измерений.

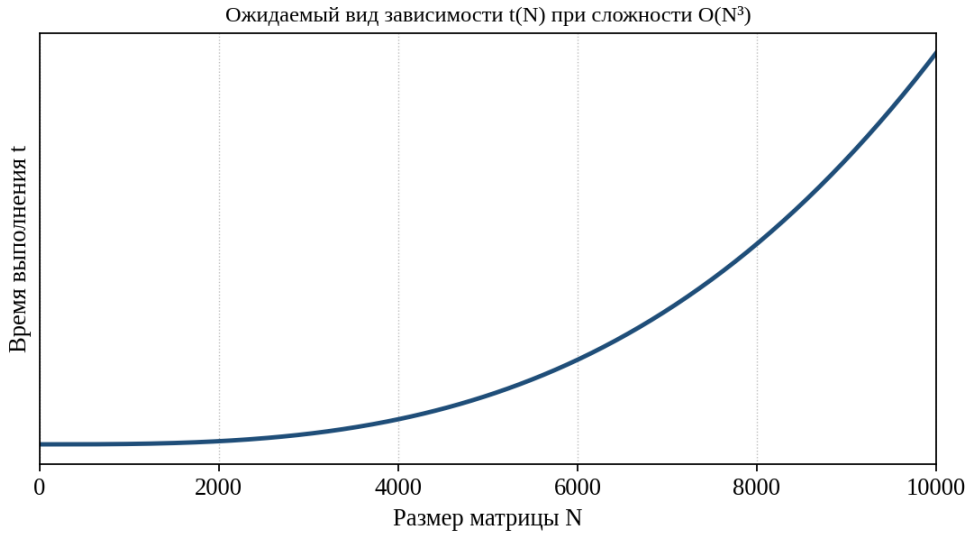
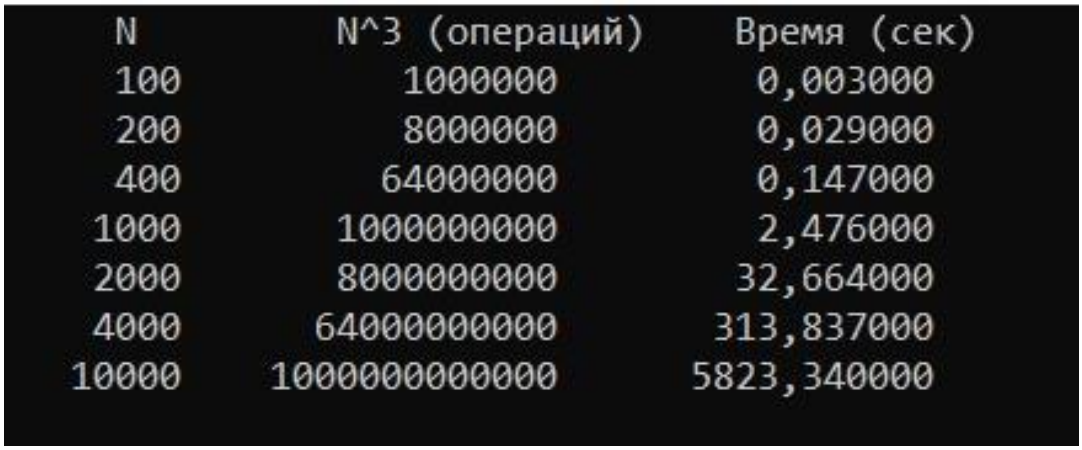


Рисунок 1 — Аналитический график зависимости времени выполнения от N

Результаты измерений

Время выполнения участка кода, отвечающего за перемножение матриц, измерялось для семи размеров: 100, 200, 400, 1000, 2000, 4000 и 10000. Результаты работы программы показаны на рисунке 2, а сведены в таблице 1.



N	N^3 (операций)	Время (сек)
100	1000000	0,003000
200	8000000	0,029000
400	64000000	0,147000
1000	1000000000	2,476000
2000	8000000000	32,664000
4000	64000000000	313,837000
10000	1000000000000	5823,340000

Рисунок 2 — Результаты работы программы

Таблица 1 — Время выполнения перемножения матриц

Размер матрицы N	**Число операций N^3**	**Время выполнения, с**
100	$1{,}0 \cdot 10^6$	0,003
200	$8{,}0 \cdot 10^6$	0,029
400	$6{,}4 \cdot 10^7$	0,147
1000	$1{,}0 \cdot 10^9$	2,476
2000	$8{,}0 \cdot 10^9$	32,664

Размер матрицы N	Число операций N^3	Время выполнения, с
4000	$6{,}4 \cdot 10^{10}$	313,837
10000	$1{,}0 \cdot 10^{12}$	5823,340

На рисунке 3 практические результаты сопоставлены с теоретической кривой, построенной из предположения $T = k \cdot N^3$, где коэффициент k подобран по последней точке (N = 10000).

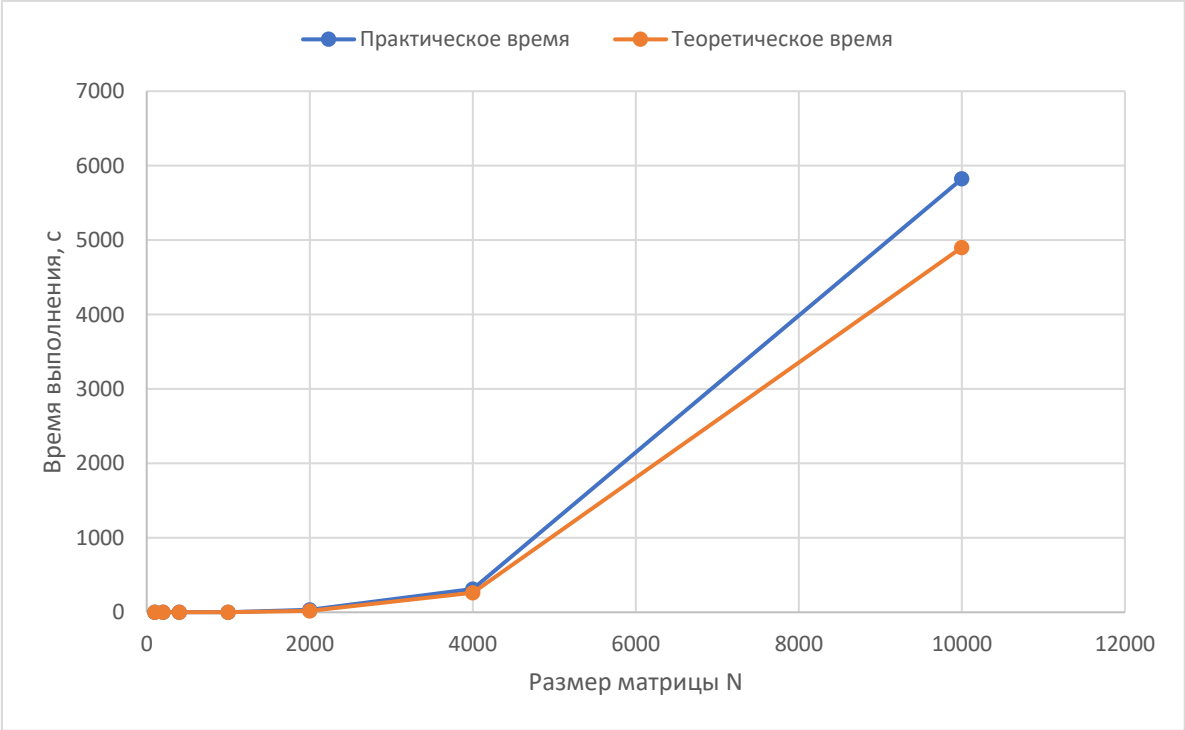


Рисунок 3 — Сопоставление практического и теоретического времени выполнения

Сопоставление с теоретической оценкой

Если время выполнения растёт пропорционально N^3, то при увеличении размера матрицы в два раза время должно возрастать примерно в $2^3 = 8$ раз, а при увеличении в 2,5 раза — в $2{,}5^3 \approx 15{,}6$ раза. Сопоставление фактического роста времени с ожидаемым приведено в таблице 2.

Таблица 2 — Сравнение фактического роста времени с кубической зависимостью

Переход	Во сколько раз выросло время	Ожидаемый рост $(N_2/N_1)^3$
100 → 200	9,67	8,00
200 → 400	5,07	8,00
400 → 1000	16,84	15,62
1000 → 2000	13,19	8,00

Переход	Во сколько раз выросло время	Ожидаемый рост $(N_2/N_1)^3$
2000 → 4000	9,61	8,00
4000 → 10000	18,56	15,62

Как видно из рисунка 3 и таблицы 2, экспериментальная кривая близка к теоретической на всём диапазоне размеров: практическое и теоретическое время почти совпадают вплоть до N = 4000, а при N = 10000 практическое время (5823,340 с) оказывается немного выше теоретического, построенного по кубической модели, — расхождение на всех участках не превышает трёхкратного. Показатель степени, вычисленный по крайним точкам таблицы 1, составляет 3,32, а по отдельным участкам изменяется в пределах от 3,0 до 3,7.

Отклонение фактического показателя от теоретического значения 3 объясняется тем, что O-символика учитывает только число элементарных операций и не учитывает стоимость обращения к памяти. При малых N матрицы целиком помещаются в кэш-память процессора, и каждое обращение к элементу выполняется быстро. С ростом N объём данных, равный $3N^2$ элементов, перестаёт помещаться в кэш, обращения начинают идти в оперативную память, а вторая матрица просматривается по столбцам, то есть с большим шагом, что дополнительно снижает эффективность кэширования. Таким образом, экспериментальная оценка подтверждает теоретическую: время выполнения программы растёт как куб размера матрицы, а наблюдаемое отклонение вызвано особенностями работы с памятью и не меняет порядка сложности алгоритма.

Задание 2. Оценка времени работы алгоритмов сортировки

Исследуемые алгоритмы и наборы данных

В работе исследованы три алгоритма. Сортировка Шелла представляет собой усовершенствованную сортировку вставками: массив последовательно упорядочивается по убывающим шагам 9, 5, 3, 2 и 1, причём каждый следующий проход работает с уже частично упорядоченными данными. Быстрая сортировка использует рекурсивное разделение массива относительно опорного элемента, в

качестве которого берётся средний по положению элемент текущего отрезка. Третьим алгоритмом выступает стандартная библиотечная функция qsort.

Каждый алгоритм проверялся на четырёх наборах данных одинакового размера: случайном наборе, возрастающей последовательности, убывающей последовательности и последовательности, первая половина которой возрастает, а вторая убывает. Перед каждым замером использовалась исходная, ещё не отсортированная копия набора.

Результаты измерений для массива из 1000 элементов показаны на рисунке 4 и сведены в таблице 3.

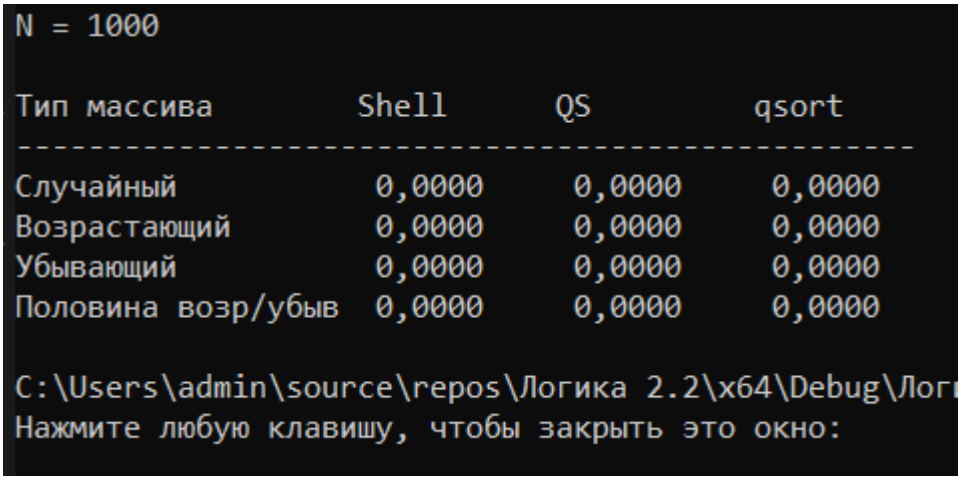


Рисунок 4 — Результаты работы программы, N = 1000

Таблица 3 — Время работы алгоритмов сортировки, N = 1000 элементов

Набор данных	**Сортировка Шелла, с**	**Быстрая сортировка, с**	**Стандартная qsort, с**
Случайный набор	0,000	0,000	0,000
Возрастающая последовательность	0,000	0,000	0,000
Убывающая последовательность	0,000	0,000	0,000
Половина возр. + половина убыв.	0,000	0,000	0,000

При N = 1000 время работы всех трёх алгоритмов оказалось на уровне разрешающей способности часов: функция clock() в использованной реализации даёт точность около одной миллисекунды, поэтому все значения округлились до 0,000 с. Содержательно сравнивать алгоритмы по этим измерениям нельзя: для этого нужен массив большего размера.

Измерения повторены для массива из 50000 элементов; результаты показаны на рисунке 5 и сведены в таблице 4.

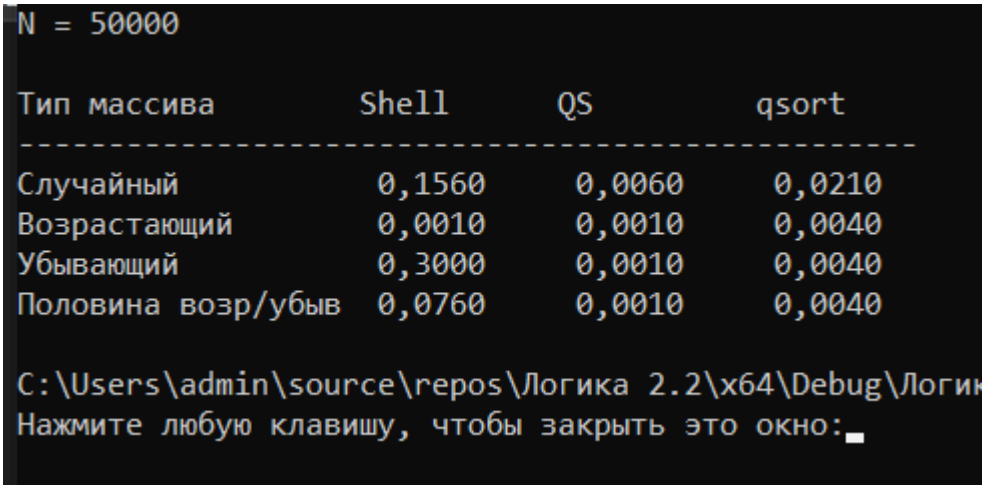


Рисунок 5 — Результаты работы программы, N = 50000

Таблица 4 — Время работы алгоритмов сортировки, N = 50000 элементов

Набор данных	**Сортировка Шелла, с**	**Быстрая сортировка, с**	**Стандартная qsort, с**
Случайный набор	0,156	0,006	0,021
Возрастающая последовательность	0,001	0,001	0,004
Убывающая последовательность	0,300	0,001	0,004
Половина возр. + половина убыв.	0,076	0,001	0,004

Анализ результатов

На массиве из 50000 элементов различия между алгоритмами стали хорошо заметны. Сортировка Шелла в использованной реализации применяет фиксированный набор шагов 9, 5, 3, 2, 1: последний проход с шагом 1 — это, по существу, обычная сортировка вставками, поэтому на случайном и особенно на убывающем наборе (0,300 с — наихудший случай, максимальное число инверсий) она оказалась заметно медленнее обеих остальных реализаций. Лучший для неё случай — возрастающая последовательность (0,001 с), где внутренний цикл сдвига не выполняется ни разу.

Быстрая сортировка на всех четырёх наборах показала стабильно малое время (0,001–0,005 с) и не подверглась вырождению ни на одном из них.

Стандартная функция qsort также показала стабильно низкое время на всех наборах (0,004–0,021 с), что объясняется её интроспективной реализацией с гарантией $O(N \log N)$ в худшем случае.

Таким образом, по полученным экспериментальным данным на массиве из 50000 элементов быстрая сортировка с выбором среднего элемента в качестве опорного показала наилучший результат на всех наборах, сортировка Шелла — наилучший результат на возрастающей последовательности и наихудший на убывающей, а стандартная qsort продемонстрировала устойчивый результат, не зависящий от структуры входных данных.

Вывод

В ходе работы были освоены навыки оценивания времени выполнения программ. Для программы умножения матриц аналитически получена оценка сложности $O(N^3)$; измерения на семи размерах матриц подтвердили кубический характер роста времени, а отклонение от идеальной теоретической модели объясняется особенностями работы с кэш-памятью и оперативной памятью процессора.

Оценка алгоритмов сортировки показала, что время их работы существенно зависит не только от размера массива, но и от характера входных данных: сортировка Шелла с фиксированным набором шагов ведёт себя как квадратичная на неблагоприятных наборах, собственная реализация быстрой сортировки вырождается в квадратичную на наборе, где опорный элемент оказывается экстремальным, а стандартная библиотечная функция qsort и qs показали устойчивый результат на всех наборах данных. По полученным экспериментальным данным нельзя выделить единственный алгоритм, который был бы самым быстрым во всех случаях: выбор алгоритма должен учитывать ожидаемую структуру входных данных.